



IMD1116

COMPUTAÇÃO DE

ALTO DESEMPENHO

MPI:

COMUNICAÇÃO COLETIVA

Prof. Samuel Xavier de Souza
Prof. Carla Santana



Método do Trapézio - MPI

Implemente, uma **versão paralela em MPI** de um programa para o cálculo de integrais definidas por meio do método do trapézio. O programa deve permitir calcular a integral de uma função $f(x)$ em um intervalo $[a,b]$, utilizando um número n de subdivisões.

O processo 0 deverá receber as áreas parciais calculadas por cada processo. O código deve ser implementado de forma a permitir execução com mais de dois processos, utilizando apenas comunicações ponto a ponto.

Você terá 15-20 minutos para implementar o programa.





Método do Trapézio - MPI

E se conseguíssemos receber tudo de uma vez?

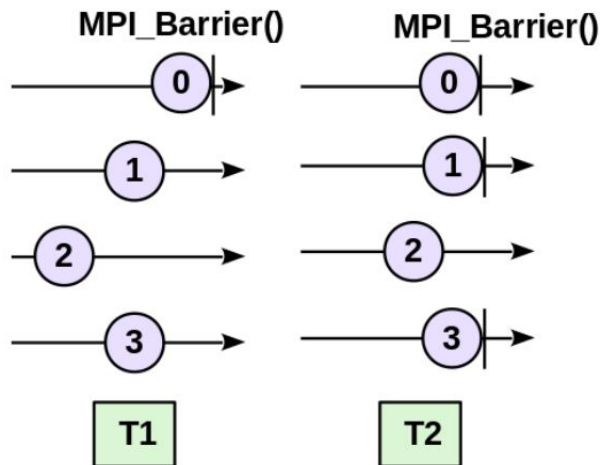


Comunicação Coletiva

Um padrão de comunicação **envolvendo todos os processos em um comunicador** é chamado de comunicação coletiva.

- Barrier
 - Broadcast
 - Scatter
 - Gather
 - Reduce
-

Comunicação Coletiva - Barrier



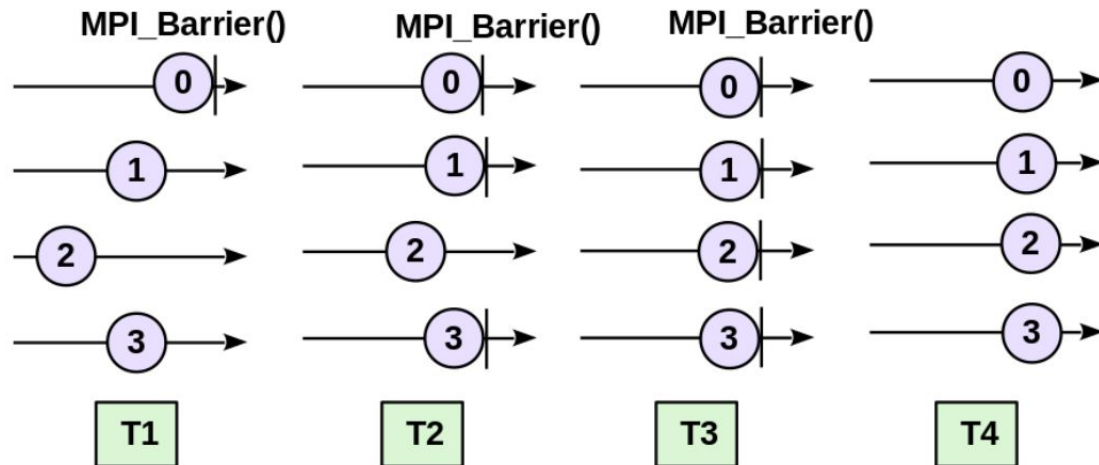
```
int main ( int argc , char * argv []) {

    int meu_ranque , num_procs ;
    MPI_Init (& argc ,& argv );
    MPI_Comm_size ( MPI_COMM_WORLD ,& num_procs );
    MPI_Comm_rank ( MPI_COMM_WORLD ,& meu_ranque );

    if ( meu_ranque == 0 ) {
        printf ( " Estou atrasado para a barreira ! \n");
        getchar ( ) ;
    }

    MPI_Barrier ( MPI_COMM_WORLD );
    printf ( " Passei da barreira . Eu sou o %d de %d, processos \n",
            meu_ranque , num_procs );
    MPI_Finalize ( ) ;
    return (0) ;
}
```

Comunicação Coletiva - Barrier



```
int main ( int argc , char * argv []) {  
  
    int meu_ranque , num_procs ;  
    MPI_Init (& argc ,& argv ) ;  
    MPI_Comm_size ( MPI_COMM_WORLD ,& num_procs ) ;  
    MPI_Comm_rank ( MPI_COMM_WORLD ,& meu_ranque ) ;  
  
    if ( meu_ranque == 0 ) {  
        printf ( " Estou atrasado para a barreira ! \n" ) ;  
        getchar ( ) ;  
    }  
  
    MPI_Barrier ( MPI_COMM_WORLD ) ;  
    printf ( " Passei da barreira . Eu sou o %d de %d, processos \n" ,  
            meu_ranque , num_procs ) ;  
    MPI_Finalize ( ) ;  
    return ( 0 ) ;  
}
```

MPI Barrier

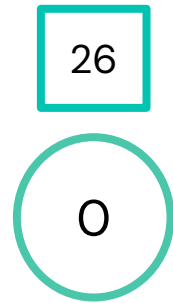
```
int MPI_Barrier(MPI_Comm com)
```

- **com**: comunicador utilizado.

Comunicação Coletiva - Broadcast

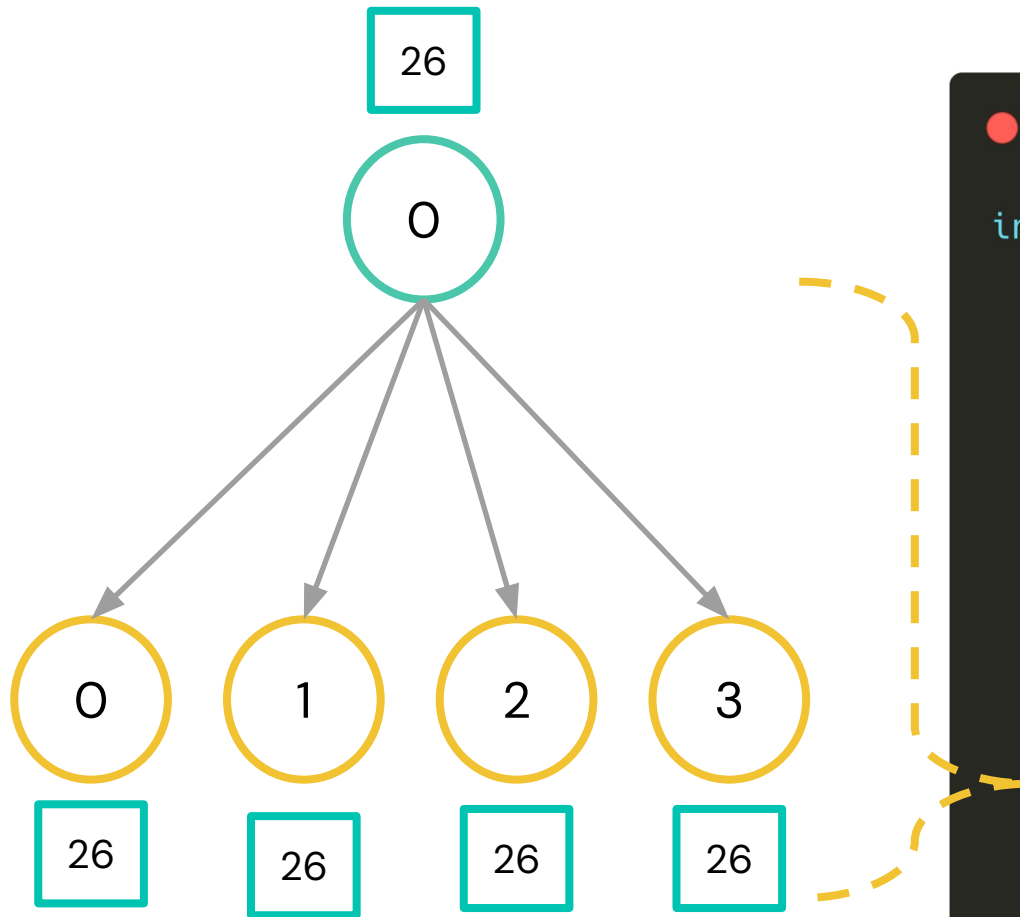
```
int main(int argc, char *argv[]) {
    int rank, size;
    int v;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    // Processo raiz define o valor
    if (rank == 0) {
        v = 26;
        printf("Processo 0 enviando valor: %d\n", v);
    }
    // Broadcast: envia de rank 0 para todos
    MPI_Bcast(&v, 1, MPI_INT, 0, MPI_COMM_WORLD);
    // Todos os processos recebem o valor
    printf("Processo %d recebeu valor: %d\n", rank, v);
    MPI_Finalize();
    return 0;
}
```


Comunicação Coletiva - Broadcast



```
int main(int argc, char *argv[]) {
    int rank, size;
    int v;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    // Processo raiz define o valor
    if (rank == 0) {
        v = 26;
        printf("Processo 0 enviando valor: %d\n", valor);
    }
    // Broadcast: envia de rank 0 para todos
    MPI_Bcast(&v, 1, MPI_INT, 0, MPI_COMM_WORLD);
    // Todos os processos recebem o valor
    printf("Processo %d recebeu valor: %d\n", rank, valor);
    MPI_Finalize();
    return 0;
}
```

Comunicação Coletiva - Broadcast



```
int main(int argc, char *argv[]) {
    int rank, size;
    int v;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    // Processo raiz define o valor
    if (rank == 0) {
        v = 26;
        printf("Processo 0 enviando valor: %d\n", v);
    }
    // Broadcast: envia de rank 0 para todos
    MPI_Bcast(&v, 1, MPI_INT, 0, MPI_COMM_WORLD);
    // Todos os processos recebem o valor
    printf("Processo %d recebeu valor: %d\n", rank, v);
    MPI_Finalize();
    return 0;
}
```

MPI Bcast

```
int MPI_Bcast(void* mensagem, int cont, MPI_Datatype tipo_mpi, int raiz, MPI_Comm com)
```

- **mensagem** : dado a ser enviado pela raiz e recebido pelos demais
 - **raiz**: processo raiz (o que está **enviando** o dado aos outros)
-

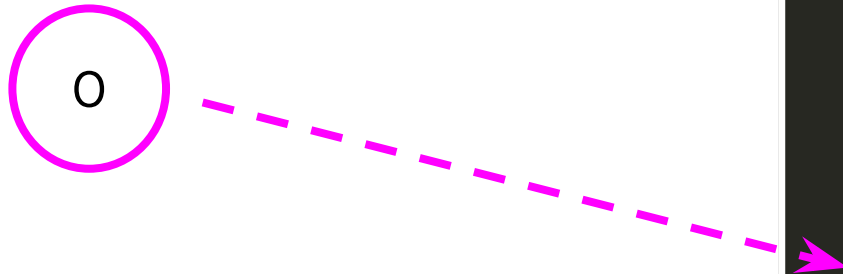
Comunicação Coletiva - Scatter

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    int dados[8];      // vetor no root
    int parte[2];      // cada processo recebe 2 elementos
    // Inicializa apenas no root
    if (rank == 0) {
        for (int i = 0; i < 8; i++) {
            dados[i] = i;
        }
        printf("Processo 0 enviando: ");
        for (int i = 0; i < 8; i++) {
            printf("%d ", dados[i]);
        }
        printf("\n");
    }

    // Distribui partes do vetor
    MPI_Scatter(dados, 2, MPI_INT,      // origem (root)
               parte, 2, MPI_INT,      // destino (cada processo)
               0, MPI_COMM_WORLD);

    // Cada processo imprime o que recebeu
    printf("Processo %d recebeu: %d %d\n", rank, parte[0], parte[1]);
    MPI_Finalize();
    return 0;
}
```

Comunicação Coletiva - Scatter

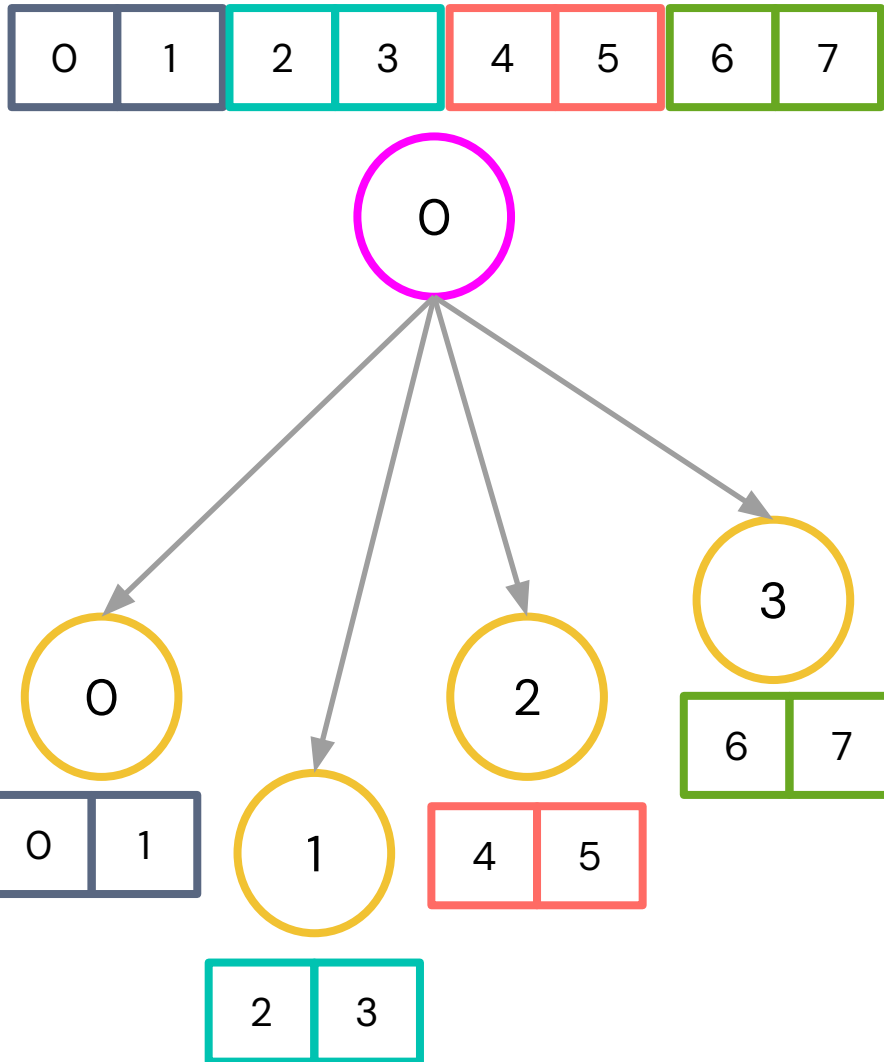


```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    int dados[8];      // vetor no root
    int parte[2];      // cada processo recebe 2 elementos
    // Inicializa apenas no root
    if (rank == 0) {
        for (int i = 0; i < 8; i++) {
            dados[i] = i;
        }
        printf("Processo 0 enviando: ");
        for (int i = 0; i < 8; i++) {
            printf("%d ", dados[i]);
        }
        printf("\n");
    }

    // Distribui partes do vetor
    MPI_Scatter(dados, 2, MPI_INT,      // origem (root)
               parte, 2, MPI_INT,      // destino (cada processo)
               0, MPI_COMM_WORLD);

    // Cada processo imprime o que recebeu
    printf("Processo %d recebeu: %d %d\n", rank, parte[0], parte[1]);
    MPI_Finalize();
    return 0;
}
```


Comunicação Coletiva - Scatter



```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    int dados[8];      // vetor no root
    int parte[2];      // cada processo recebe 2 elementos
    // Inicializa apenas no root
    if (rank == 0) {
        for (int i = 0; i < 8; i++) {
            dados[i] = i;
        }
        printf("Processo 0 enviando: ");
        for (int i = 0; i < 8; i++) {
            printf("%d ", dados[i]);
        }
        printf("\n");
    }

    // Distribui partes do vetor
    MPI_Scatter(dados, 2, MPI_INT,      // origem (root)
               parte, 2, MPI_INT,      // destino (cada processo)
               0, MPI_COMM_WORLD);

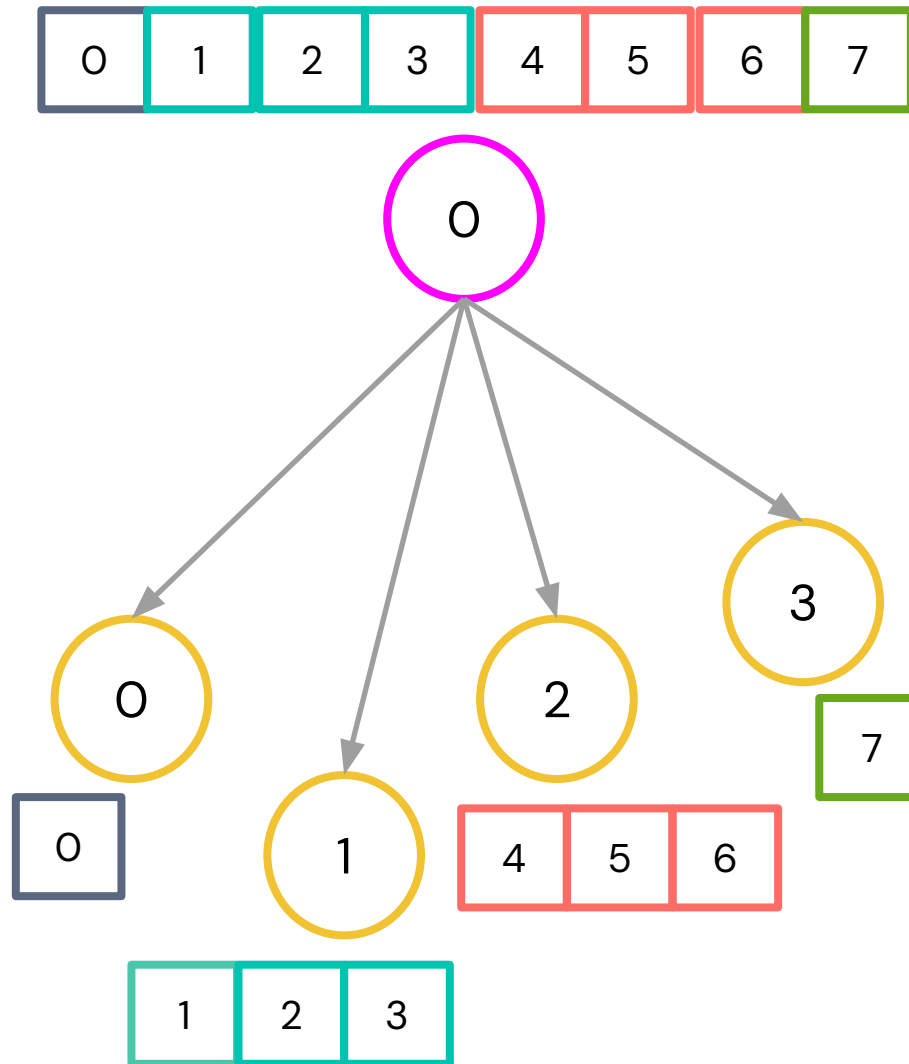
    // Cada processo imprime o que recebeu
    printf("Processo %d recebeu: %d %d\n", rank, parte[0], parte[1]);
    MPI_Finalize();
    return 0;
}
```

MPI_Scatter

```
int MPI_Scatter( void* vet_envia,      int cont_envia,      MPI_Datatype tipo_envia,  
                void* vet_recebe,    int cont_recebe,    MPI_Datatype tipo_recebe,  
                int raiz, MPI_Comm com)
```

- **vet_envia** : dado com o conteúdo que será enviado
 - **cont_envia** : tamanho do dado que será enviado. O conteúdo de **vet_envia** é dividido pela quantidade de processos, cada um consistindo de **cont_envia** itens. **cont_envia** não é do tamanho do **vet_envia** , mas sim o número de itens enviados do processo raiz para cada um dos processos.
 - **tipo_envia** : o tipo MPI dos dados a serem enviados
 - **vet_recebe** : dado com o conteúdo que será recebido
 - **cont_recebe** : tamanho do dado que será recebido
 - **tipo_recebe** : o tipo MPI dos dados a serem recebido
 - **raiz**: processo raiz (o que está **enviando** o dado aos outros)
-

Comunicação Coletiva - Scatterv



MPI_Scatterv

```
int MPI_Scatter( void* vet_envia,    int cont_envia,    MPI_Datatype tipo_envia,  
                void* vet_recebe,   int cont_recebe,   MPI_Datatype tipo_recebe,  
                int raiz, MPI_Comm com)
```

```
int MPI_Scatterv( void* vet_envia,  int *cont_envia, int *desloc, MPI_Datatype tipo_envia,  
                 void* vet_recebe,  int cont_recebe, MPI_Datatype tipo_recebe,  
                 int raiz, MPI_Comm com)
```

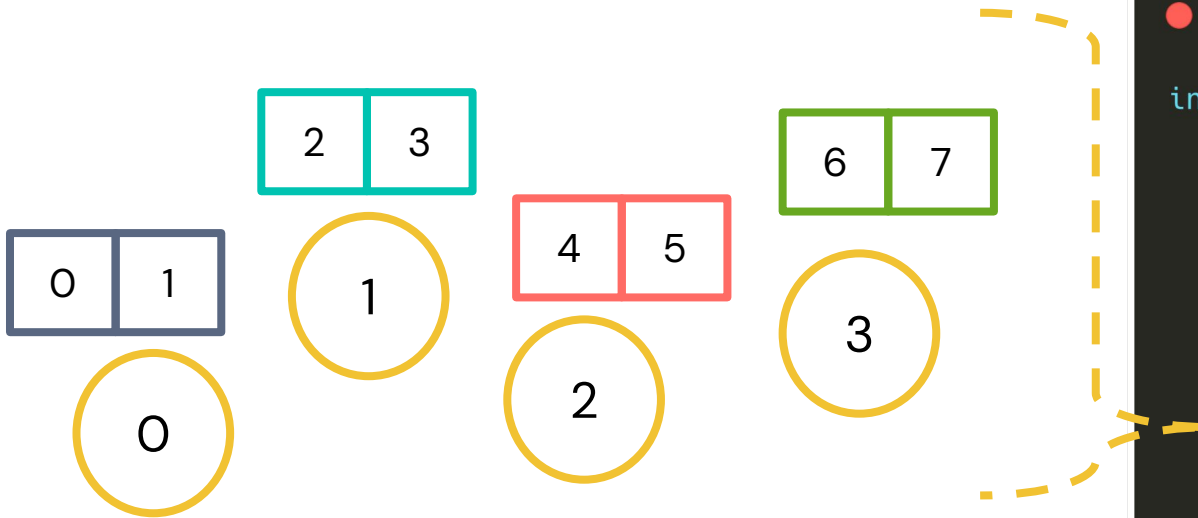
- **vet_envia** : dado com o conteúdo que será enviado
 - ***cont_envia** : quantos elementos enviar para o processo
 - **desloc** : deslocamento (posição inicial no vetor)
 - **vet_recebe** : dado com o conteúdo que será recebido
 - **cont_recebe** : tamanho do dado que será recebido pelo processo
-

Comunicação Coletiva - Gather

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    int parte[2];           // dados locais de cada processo
    int resultado[8];       // apenas no root
    // Cada processo cria seus próprios dados
    for (int i = 0; i < 2; i++) {
        parte[i] = rank * 2 + i;
    }
    printf("Processo %d enviando: %d %d\n", rank, parte[0], parte[1]);
    // Junta tudo no processo 0
    MPI_Gather(parte, 2, MPI_INT, // envio (cada processo)
              resultado, 2, MPI_INT, // recepção (root)
              0, MPI_COMM_WORLD);

    // Apenas o root imprime o resultado final
    if (rank == 0) {
        printf("Processo 0 recebeu: ");
        for (int i = 0; i < size * 2; i++) {
            printf("%d ", resultado[i]);
        }
        printf("\n");
    }
    MPI_Finalize();
    return 0;
}
```

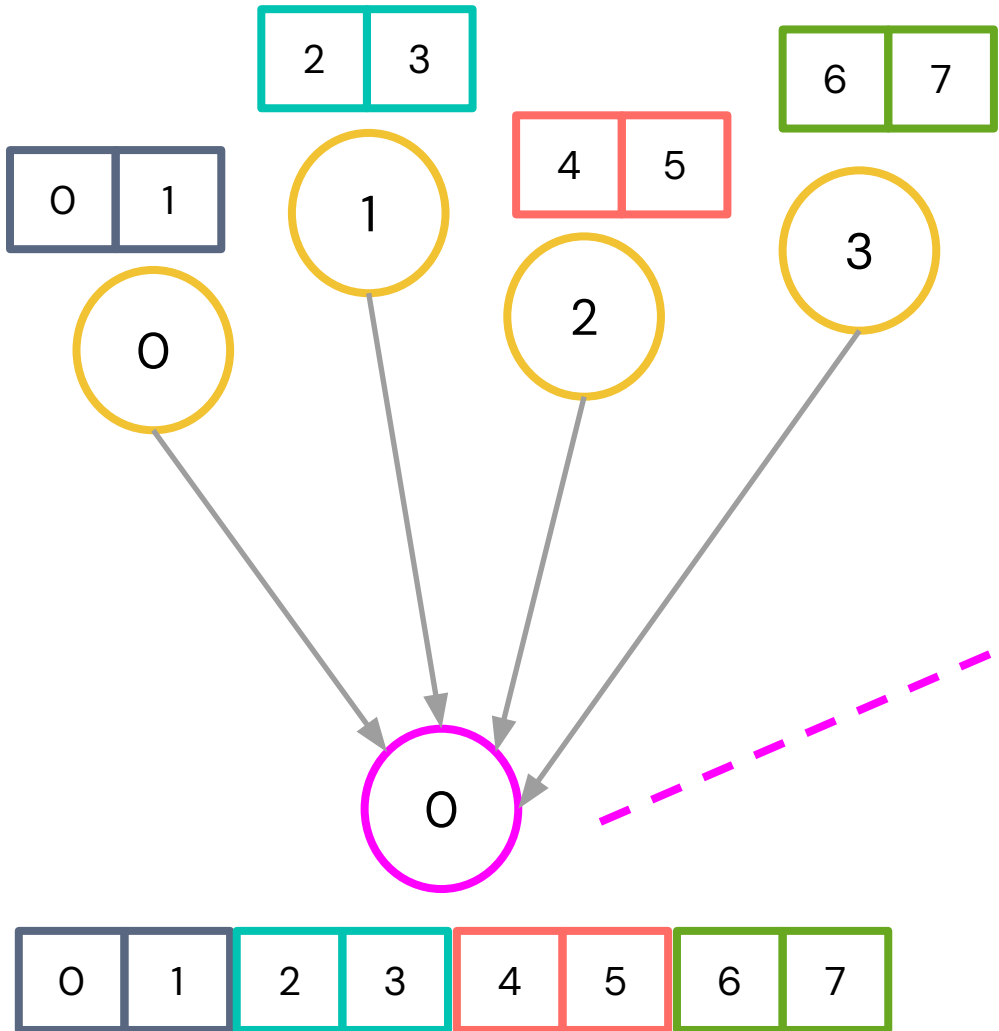
Comunicação Coletiva - Gather



```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    int parte[2];           // dados locais de cada processo
    int resultado[8];       // apenas no root
    // Cada processo cria seus próprios dados
    for (int i = 0; i < 2; i++) {
        parte[i] = rank * 2 + i;
    }
    printf("Processo %d enviando: %d %d\n", rank, parte[0], parte[1]);
    // Junta tudo no processo 0
    MPI_Gather(parte, 2, MPI_INT, // envio (cada processo)
              resultado, 2, MPI_INT, // recepção (root)
              0, MPI_COMM_WORLD);

    // Apenas o root imprime o resultado final
    if (rank == 0) {
        printf("Processo 0 recebeu: ");
        for (int i = 0; i < size * 2; i++) {
            printf("%d ", resultado[i]);
        }
        printf("\n");
    }
    MPI_Finalize();
    return 0;
}
```


Comunicação Coletiva - Gather



```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    int parte[2];           // dados locais de cada processo
    int resultado[8];       // apenas no root
    // Cada processo cria seus próprios dados
    for (int i = 0; i < 2; i++) {
        parte[i] = rank * 2 + i;
    }
    printf("Processo %d enviando: %d %d\n", rank, parte[0], parte[1]);
    // Junta tudo no processo 0
    MPI_Gather(parte, 2, MPI_INT, // envio (cada processo)
              resultado, 2, MPI_INT, // recepção (root)
              0, MPI_COMM_WORLD);

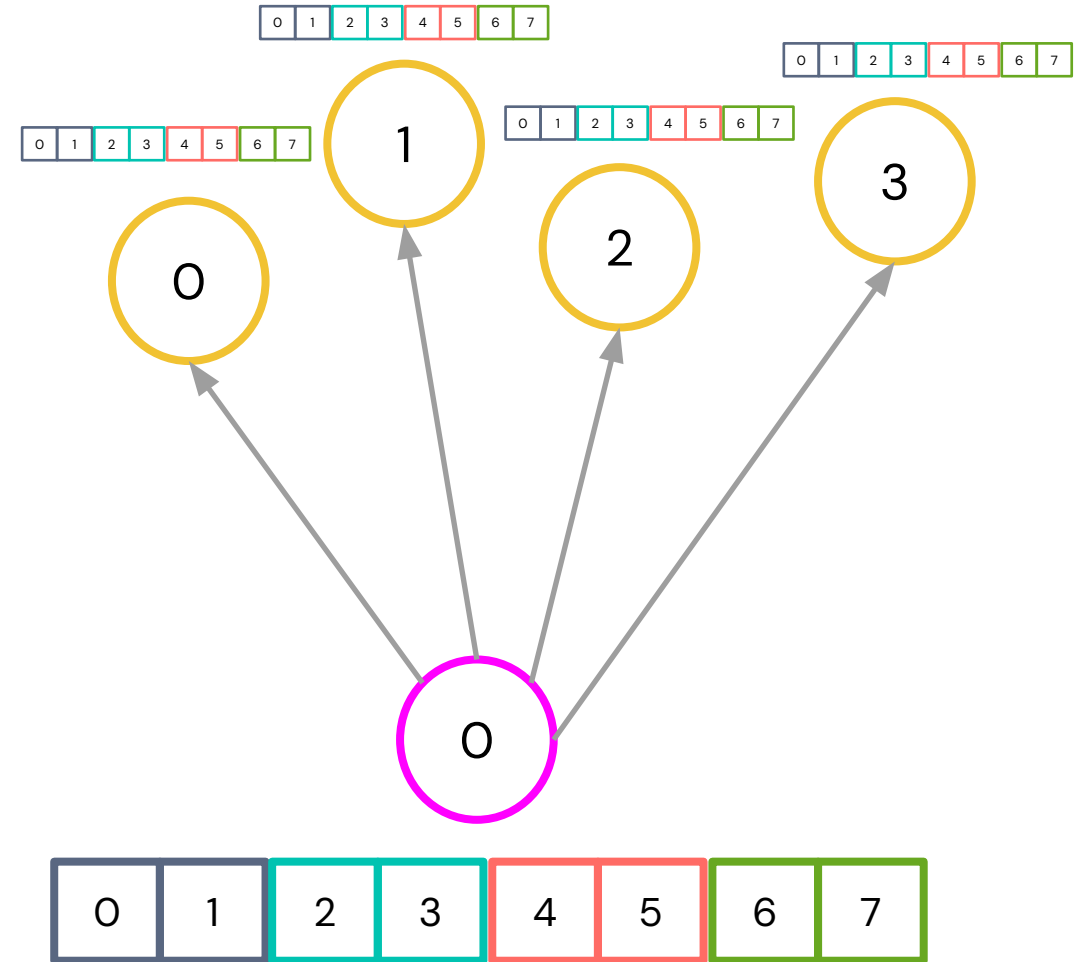
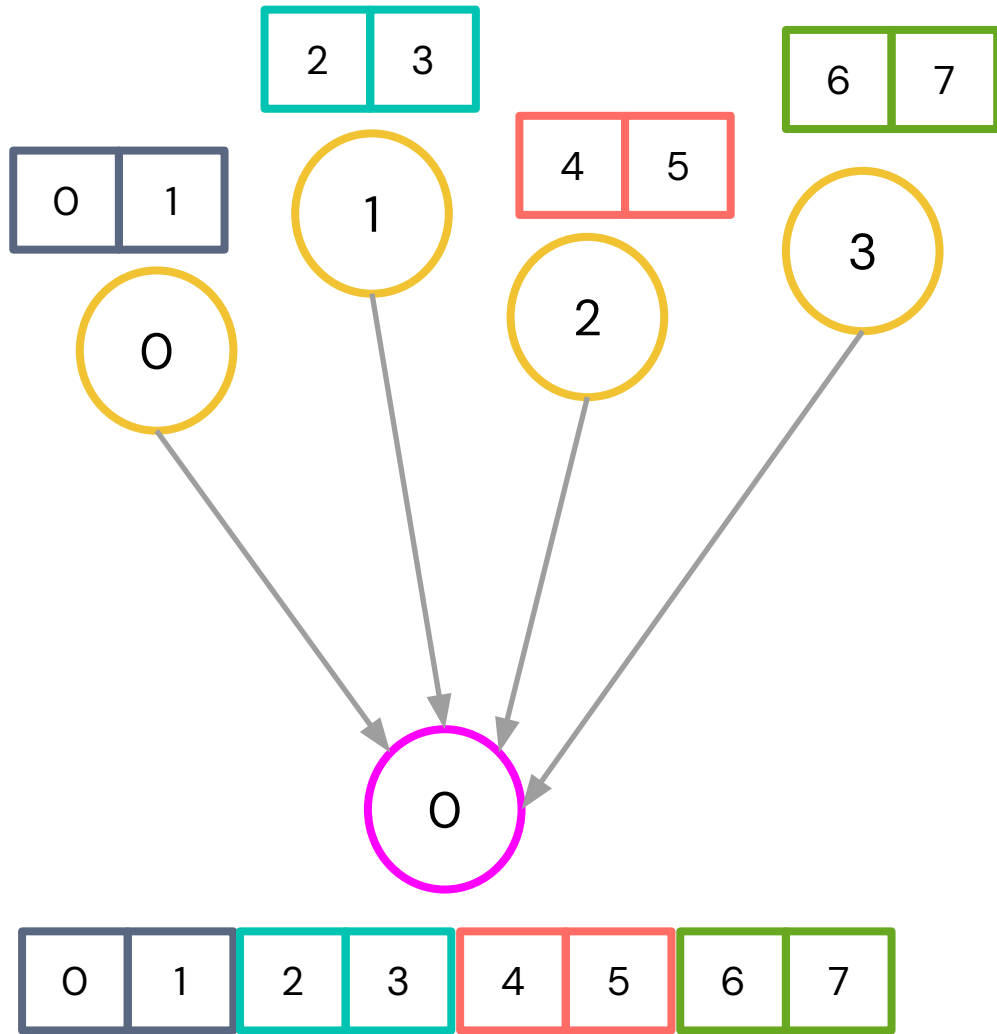
    // Apenas o root imprime o resultado final
    if (rank == 0) {
        printf("Processo 0 recebeu: ");
        for (int i = 0; i < size * 2; i++) {
            printf("%d ", resultado[i]);
        }
        printf("\n");
    }
    MPI_Finalize();
    return 0;
}
```

MPI_Gather

```
int MPI_Gather( void* vet_envia,      int cont_envia,      MPI_Datatype tipo_envia,  
               void* vet_recebe,    int cont_recebe,    MPI_Datatype tipo_recebe,  
               int raiz, MPI_Comm com)
```

- **vet_envia** : dado com o conteúdo que será enviado
 - **cont_envia** : tamanho do dado que será enviado
 - **tipo_envia** : o tipo MPI dos dados a serem enviados
 - **vet_recebe** : dado com o conteúdo que será recebido
 - **cont_recebe** : tamanho do dado que será recebido. O conteúdo de **vet_recebe** é concatenado pelos dados recebidos dos processos, cada dado consistindo de **cont_recebe** itens. **cont_recebe** não é do tamanho do **vet_recebe** , mas sim o número de itens enviados do processo raiz para cada um dos processos.
 - **tipo_recebe** : o tipo MPI dos dados a serem recebido
 - **raiz**: processo raiz (o que está **recebendo** o dado aos outros)
-

Comunicação Coletiva - Allgather



MPI_Allgather

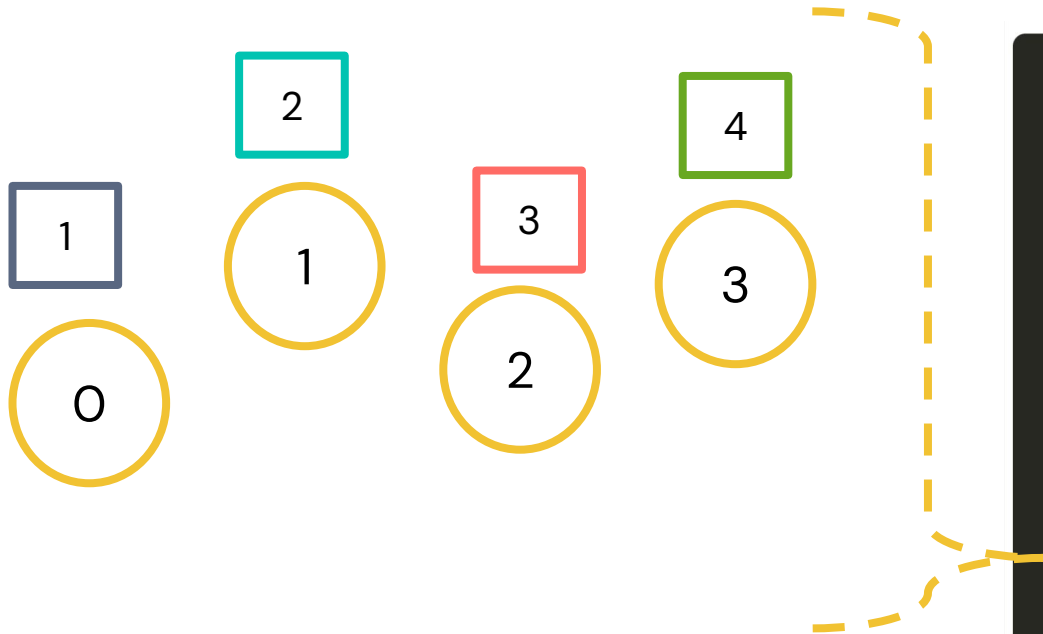
```
int MPI_Allgather(void* vet_envia,      int cont_envia,      MPI_Datatype tipo_envia,  
                  void* vet_recebe,    int cont_recebe,    MPI_Datatype tipo_recebe,  
                  MPI_Comm com)
```

- **vet_envia** : dado com o conteúdo que será enviado
 - **cont_envia** : tamanho do dado que será enviado
 - **tipo_envia** : o tipo MPI dos dados a serem enviados
 - **vet_recebe** : dado com o conteúdo que será recebido
 - **cont_recebe** : tamanho do dado que será recebido. O conteúdo de **vet_recebe** é concatenado pelos dados recebidos dos processos, cada dado consistindo de **cont_recebe** itens. **cont_recebe** não é do tamanho do **vet_recebe** , mas sim o número de itens enviados do processo raiz para cada um dos processos.
 - **tipo_recebe** : o tipo MPI dos dados a serem recebido
 - ~~**raiz**: processo raiz (o que está **recebendo** o dado aos outros)~~
-

Comunicação Coletiva - Reduce

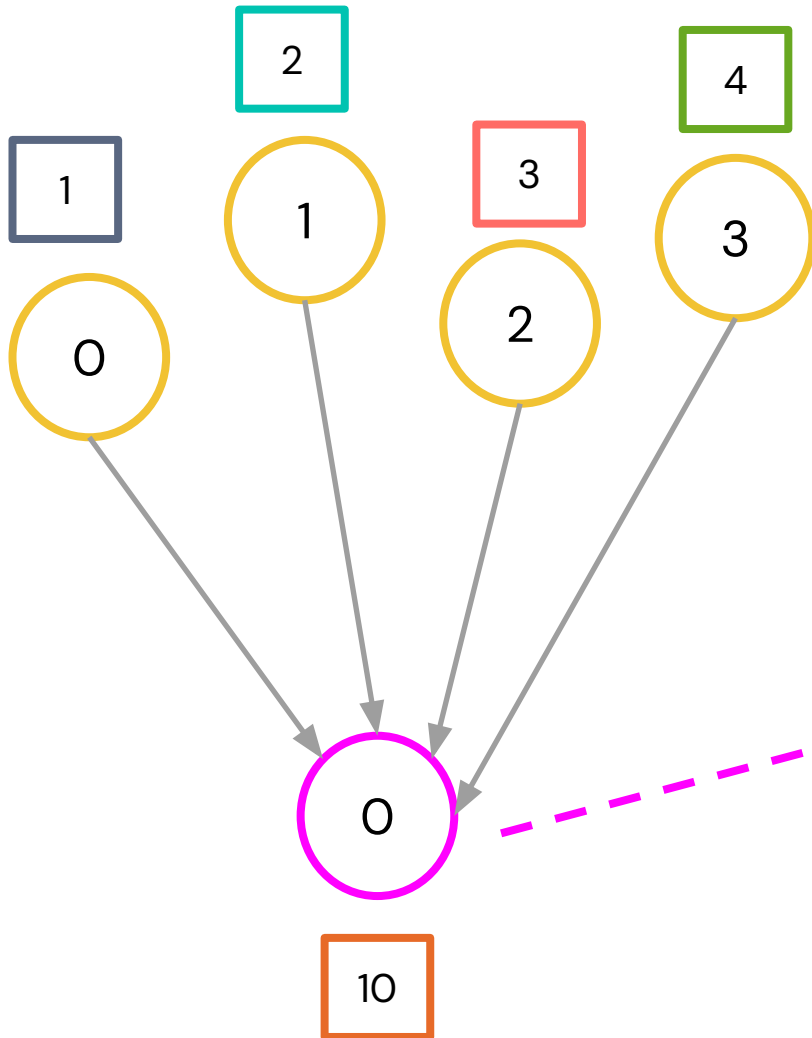
```
int main(int argc, char *argv[]) {
    int rank, size;
    int valor_local;
    int soma_total;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    // Cada processo define seu valor (ex: rank + 1)
    valor_local = rank + 1;
    printf("Processo %d tem valor: %d\n", rank, valor_local);
    // Redução: soma todos os valores no processo 0
    MPI_Reduce(&valor_local, &soma_total, 1, MPI_INT,
               MPI_SUM, 0, MPI_COMM_WORLD);
    // Apenas o root imprime o resultado
    if (rank == 0) {
        printf("Soma total = %d\n", soma_total);
    }
    MPI_Finalize();
    return 0;
}
```

Comunicação Coletiva - Reduce



```
int main(int argc, char *argv[]) {
    int rank, size;
    int valor_local;
    int soma_total;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    // Cada processo define seu valor (ex: rank + 1)
    valor_local = rank + 1;
    printf("Processo %d tem valor: %d\n", rank, valor_local);
    // Redução: soma todos os valores no processo 0
    MPI_Reduce(&valor_local, &soma_total, 1, MPI_INT,
               MPI_SUM, 0, MPI_COMM_WORLD);
    // Apenas o root imprime o resultado
    if (rank == 0) {
        printf("Soma total = %d\n", soma_total);
    }
    MPI_Finalize();
    return 0;
}
```


Comunicação Coletiva - Reduce



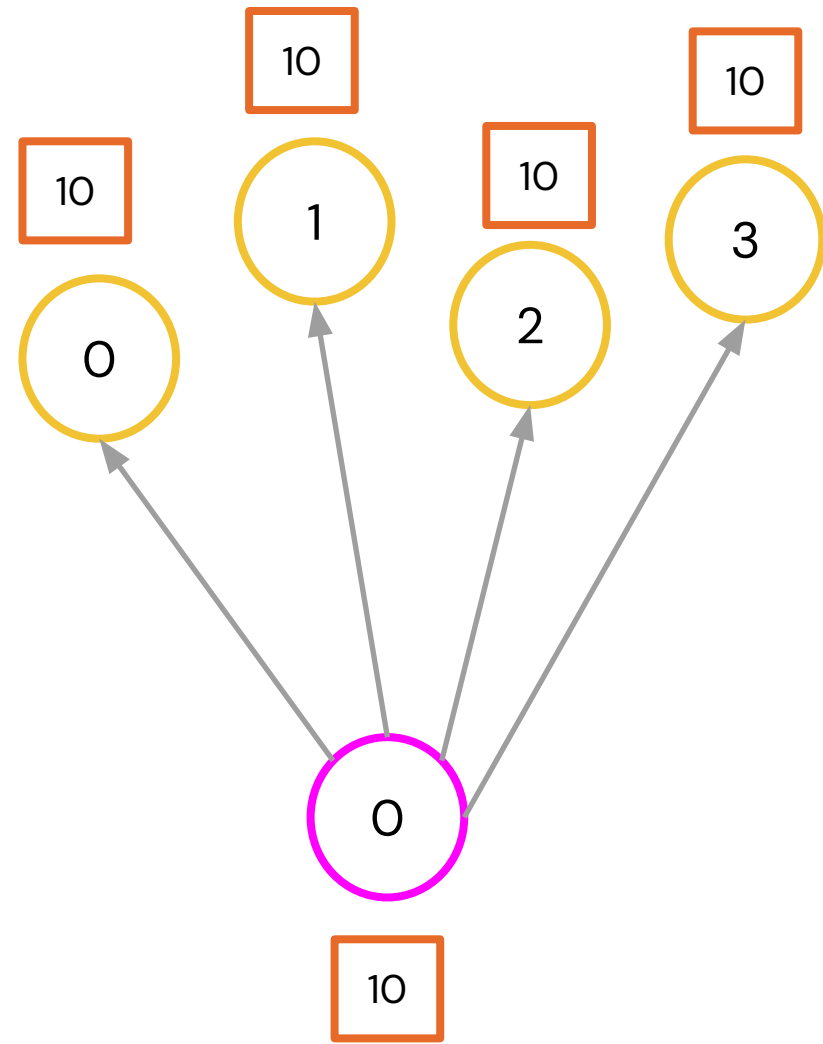
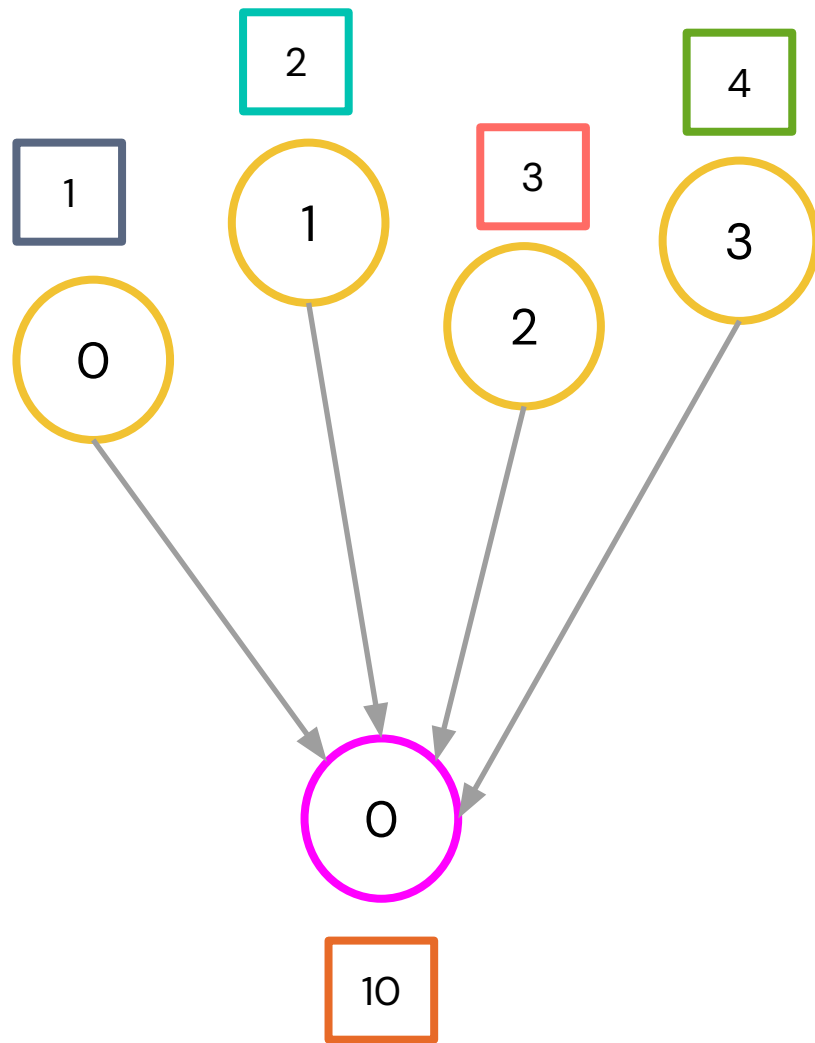
```
int main(int argc, char *argv[]) {
    int rank, size;
    int valor_local;
    int soma_total;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    // Cada processo define seu valor (ex: rank + 1)
    valor_local = rank + 1;
    printf("Processo %d tem valor: %d\n", rank, valor_local);
    // Redução: soma todos os valores no processo 0
    MPI_Reduce(&valor_local, &soma_total, 1, MPI_INT,
               MPI_SUM, 0, MPI_COMM_WORLD);
    // Apenas o root imprime o resultado
    if (rank == 0) {
        printf("Soma total = %d\n", soma_total);
    }
    MPI_Finalize();
    return 0;
}
```

MPI Reduce

```
int MPI_Reduce(void* vet_envia, void* vet_recebe, int cont,  
               MPI_Datatype tipo, MPI_Op oper, int raiz, MPI_Comm com)
```

- **vet_envia** : dado com o conteúdo que será enviado
 - **vet_recebe** : dado com o conteúdo que será recebido
 - **cont**: tamanho do dado que será enviado
 - **tipo**: o tipo MPI dos dados a serem enviados
 - **oper**: o tipo de operação (ex.: MPI_MAX, MPI_SUM, MPI_MIN ...)
 - **raiz**: processo raiz (o que está **recebendo** o dado aos outros)
-

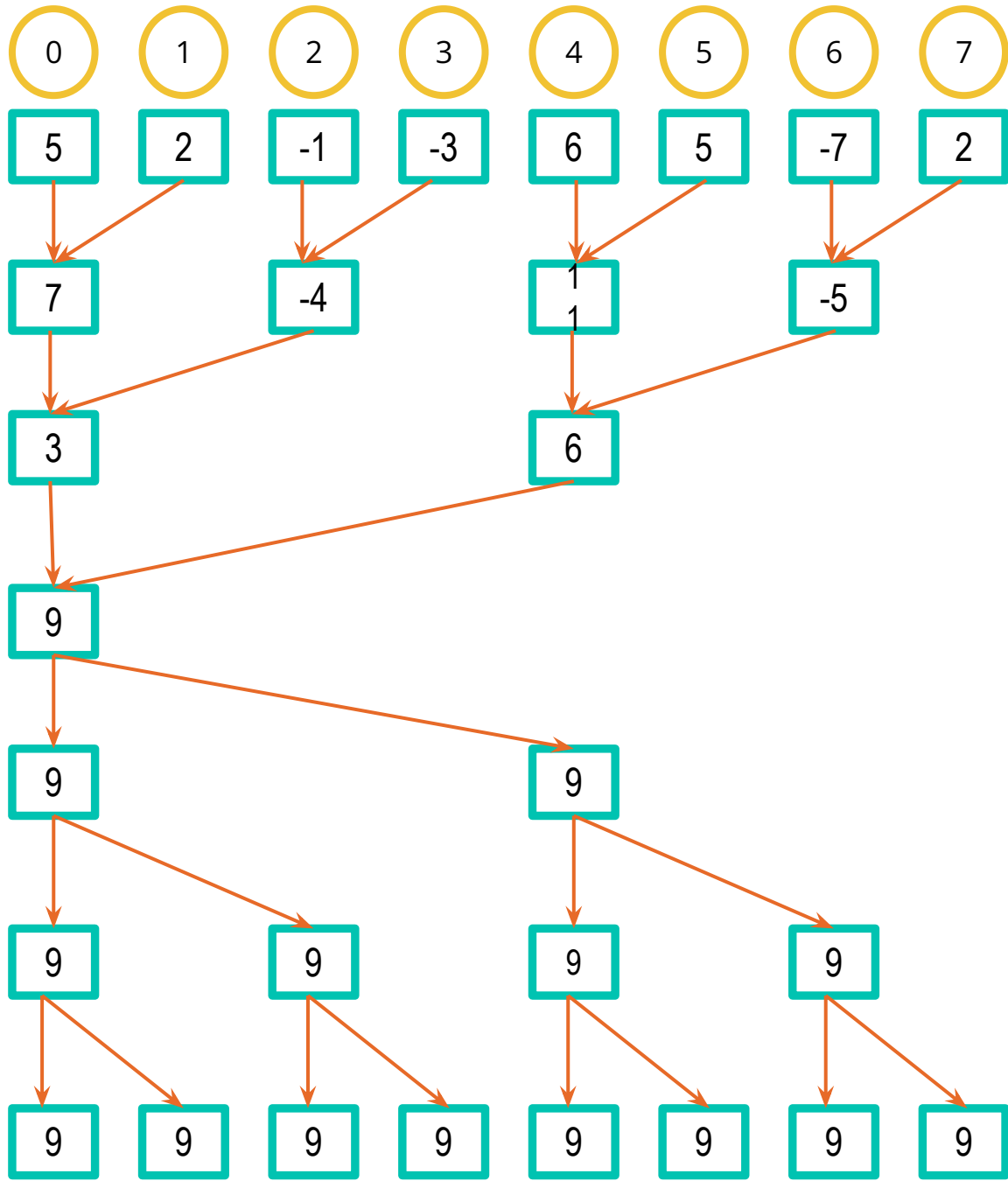
Comunicação Coletiva - Allreduce



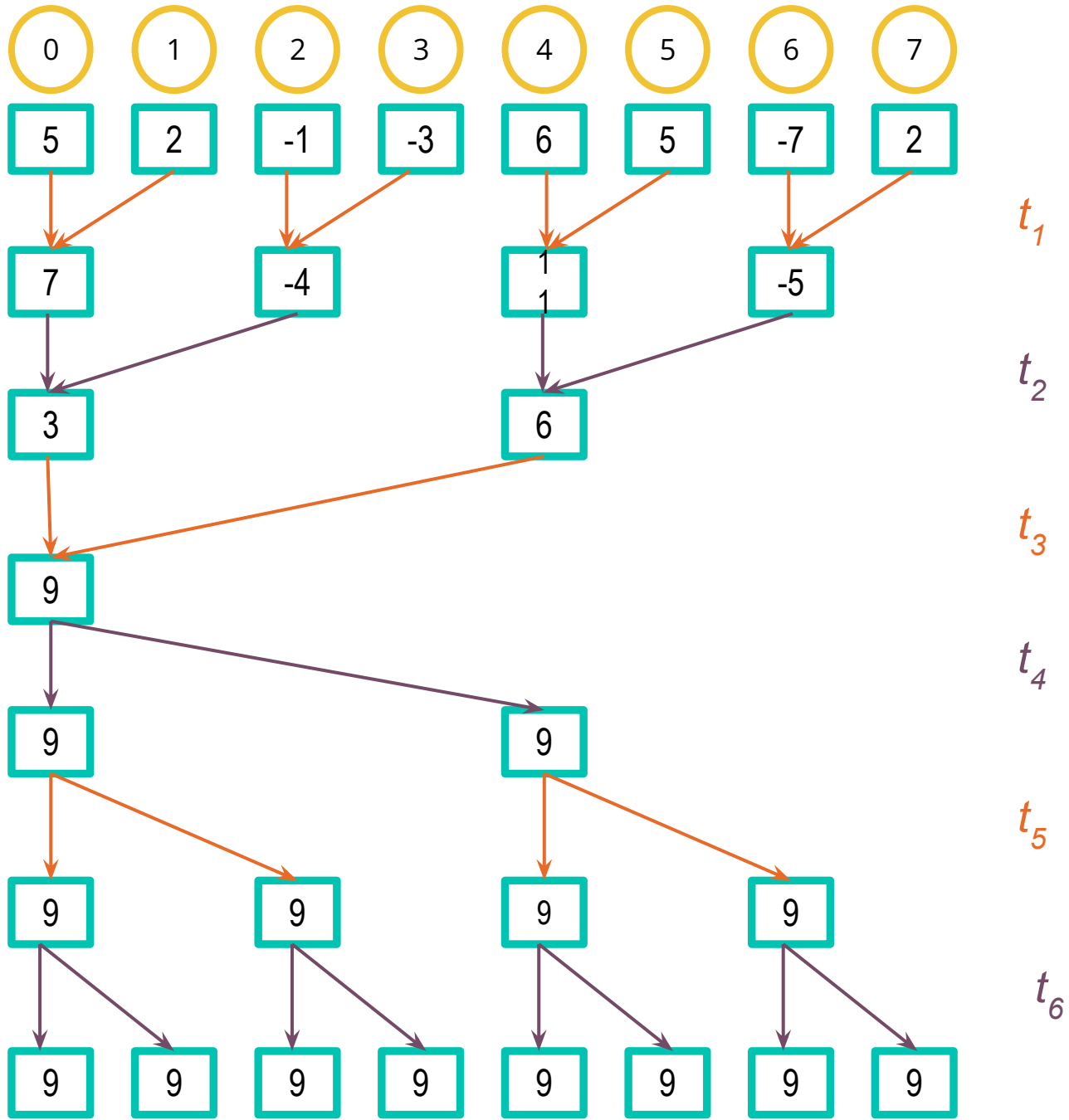
MPI Allreduce

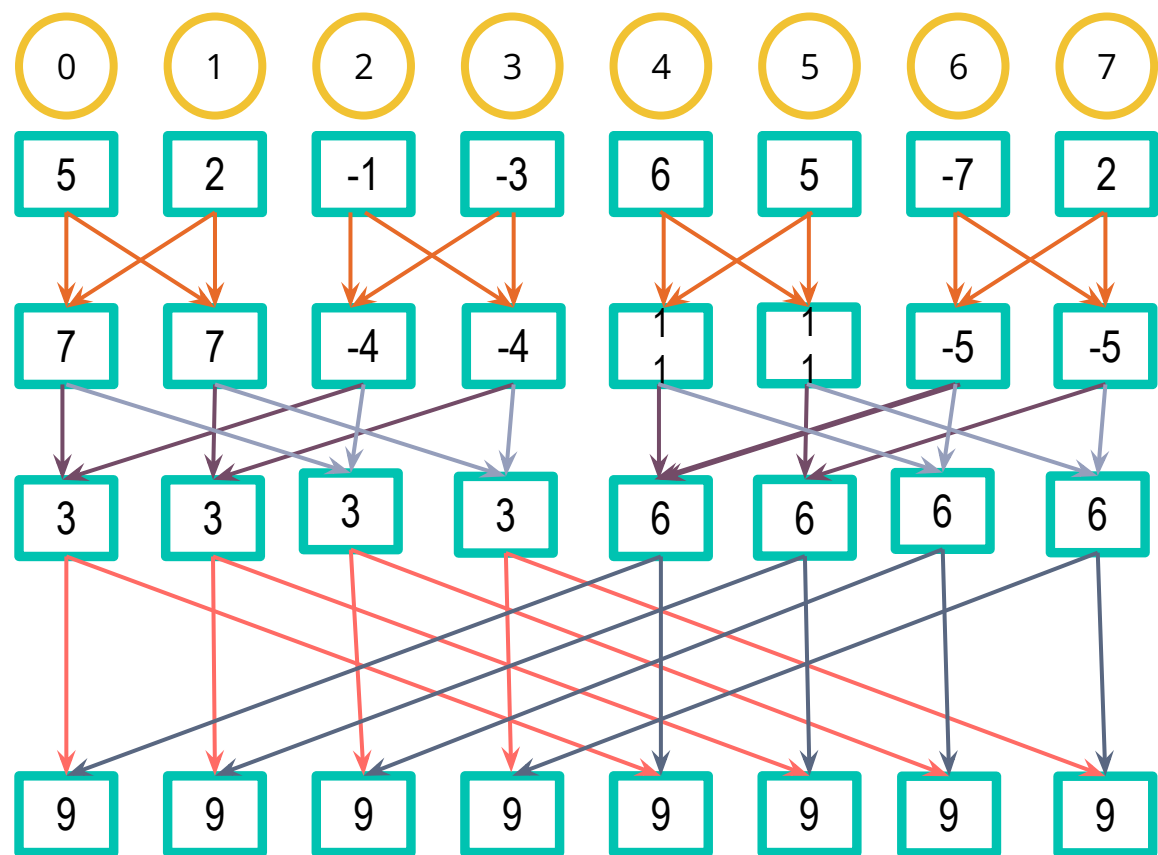
```
int MPI_Allreduce(void* vet_envia, void* vet_recebe, int cont,  
                 MPI_Datatype tipo, MPI_Op oper, MPI_Comm com)
```

- **vet_envia** : dado com o conteúdo que será enviado
 - **vet_recebe** : dado com o conteúdo que será recebido
 - **cont**: tamanho do dado que será enviado
 - **tipo**: o tipo MPI dos dados a serem enviados
 - **oper**: o tipo de operação (ex.: MPI_MAX, MPI_SUM, MPI_MIN ...)
 - ~~**raiz**: processo raiz (o que está **recebendo** o dado aos outros)~~
-



Uma soma global seguida pela distribuição do resultado.





t_1

t_2

t_3

Soma global usando estrutura borboleta

Método do Trapézio Comunicação Coletiva

Atv Extra - Implemente, uma **versão paralela em MPI** de um programa para o cálculo de integrais definidas por meio do método do trapézio. O programa deve permitir calcular a integral de uma função $f(x)$ em um intervalo $[a,b]$, utilizando um número n de subdivisões.

O processo 0 deverá receber as áreas parciais calculadas por cada processo. O código deve ser implementado de forma a permitir execução com mais de dois processos, podendo usar comunicações coletiva .

Atividade para ser entregue até **18:20**.



PROGRAMAÇÃO EM MEMÓRIA DISTRIBUÍDA

Tarefa 17:

Implemente um programa MPI que calcule o produto $y=A \cdot x$, onde **A é uma matriz $M \times N$** e **x é um vetor de tamanho N**.

Divida a matriz A por linhas entre os processos com MPI_Scatter, e distribua o vetor x inteiro com MPI_Bcast. Cada processo deve calcular os elementos de y correspondentes às suas linhas e enviá-los de volta ao processo 0 com MPI_Gather.

Compare os tempos com diferentes tamanhos de matriz e número de processos.

Fazer avaliação de eficiência e speedup a medida que aumenta o tamanho da matriz e quantidade de processos.

Referências

